

ECM1414 Algorithms and Data Structures Coursework

Documentation/Report (Group 14)

1. Introduction

1.1 The Problem The task is to create an event planner system. Given certain restrictions on the budget and time you have available, the task is to create an optimisation algorithm that selects a particular sequence of events that maximises the enjoyment value.

For each event, there are multiple associated values:

- Time – the amount of time required to complete the event
- Cost – the cost associated with completing the event
- Enjoyment – a value indicating the “enjoyment” obtained from completing the task

We were given a range of inputs to pass into the algorithm, ranging from 5 available events to choose from, all the way to 1000.

1.2 Objectives

The objectives of the coursework are to understand and develop both a brute force approach and dynamic approach to solve this optimisation problem. For a range of scenarios/inputs, we aimed to create an efficient, dynamic algorithm which is capable of handling both small and much larger inputs without presenting an unreasonable loss in performance.

This coursework asks that you aim to consider only one of the two constraints, being time and money. For our algorithms, they consider only the available budget and do not take time into account as a limiting factor.

2. Design and Approach

2.1 Program Structure

The program is structured into 3 different python files. These files are as follows:

- Event_Planner_Bruteforce_Money.py - this file contains all the logic required to run the brute-force approach algorithm
- Event_Planner_Dynamic_Money.py – this file contains all the logic required to run the dynamic approach algorithm

- Utilities.py - this file contains useful functions utilised in both main program files as to reduce redundancy of code and to make the initialisation of either file simpler. These useful functions mostly involve reading input files and putting them into a format which can be interpreted by the program more easily. By importing this file as a module, the other files can make use of this functionality when it is needed.

Dividing the program into 3 distinct files makes it much easier to read, understand and debug. The Utilities.py file is particularly good practice as well, providing functionality that both main files require whilst only writing it once.

All the files are entirely functional, meaning that the whole program essentially runs as a result of function calls, whether that is a call to the main function of the file, or more frequently needed functions provided by importing the Utilities.py module.

Additionally, the program crucially makes use of itertools module. This module is particularly useful for efficiently iterating through large combinations of elements, which is exactly what it is used for in this program. It was vital in making both algorithms more efficient.

Further modules used include NumPy, which is sometimes used throughout the program when mathematical operations are required. Using NumPy for these calculations not only often simplifies the process with a singular function (for example, summing values in a set of arrays) but also increases efficiency.

The time module is also used to keep track of the execution time, which is a crucial detail printed to the command line at the end of both algorithms.

2.2 Implemented Data Structures

Both the brute-force approach and the dynamic approach mainly utilise arrays. These vary in dimensionality and are both explicitly used by both algorithms.

In the brute-force approach, an array is used to store valid combinations of events. We chose to use an array here as it was the simplest and most efficient method available to contain the data that we needed to process; other data structures would risk introducing unnecessary complexity.

In the dynamic approach, the 2D array is much more complex, being used as a data field to contain and calculate the best combinations of events which are permitted by the given restraints for a particular scenario/input. Here, a 2D array was specifically needed to implement our planned dynamic approach – no other data structure like a stack or dictionary would be able to give us the same functionality that we needed.

3. Algorithms

3.1 The Brute-Force Approach

3.1.1 Algorithm Explanation

Firstly, the algorithm uses one of the Utilities.py functions to initialise the starting variables, such as the available events and maximum budget. Next, the algorithm will, with the help of the itertools module, create every combination of events from the ones available. Checking these combinations against the restraint, in consideration being the max budget, they will be added to a new list if they are considered “valid”, meaning that the summated cost that a given combination of events requires does not exceed the stated restraints. Finally, the algorithm will find the valid event combination that yields the greatest enjoyment value, which is then printed to the command line in a readable format, as the final solution, alongside other information such as the total enjoyment obtained, the amount of resources used to achieve this and the execution time. If there are multiple solutions, only one is returned.

3.1.2 Pseudocode

Below is a screenshot of the pseudocode for the brute-force approach algorithm, as to provide an alternative representation of how it works:

```

1  function Event_Planner_Bruteforce
2
3      function validset
4
5          set_sum is sum of array
6
7          if set_sum[money] is less than or equal to max_budget
8
9              return True
10
11          END if
12          else:
13
14              return False
15
16          END else
17
18      END function validset
19
20
21      for i in range(1, number_of_events + 1)
22
23          for subset in itertools.combinations(event_list, i)
24
25              if validset(subset) is True
26
27                  event_combo_list is event_combo_list + subset
28
29              END if
30
31          END for
32
33      END for
34
35
36      for i in range(0, event_combo_list_length)
37
38          info is sum(event_combo_list[i])
39
40          subsets_with_details.add(event_list, info)
41
42      END for
43
44      subsets_with_details is subsets_with_details + [event_combo_list[i],info]
45
46      best_combo_numeric is max of: subsets_with_details[enjoyment]
47
48      return best_combo, details
49
50
51  END function Event_Planner_Bruteforce

```

3.1.3 Time/Space Complexity

The time complexity of the brute-force approach is $O(2^n)$. As this is an exponential time complexity, it is incredibly inefficient and scales very poorly. With input_large.txt, which only has 25 available events to choose from, this method took 6 minutes and 45 seconds to execute, which is incredibly unreasonable and would be nowhere near efficient enough to match modern industry standards. The time complexity of $O(2^n)$ is found by considering how the number of combinations of events scales as more events become available – every time a new event is available to choose from, the number of possible combinations increases by a factor of 2, hence giving it the exponential time complexity that it has. The brute-force approach is not effective.

The space complexity of the brute-force approach is $O(2^n)$. This is because the space/memory used by the algorithm grows exponentially with the number of available

events. The greatest impact on memory use with this particular approach is the initial list of every possible combination of events. Hence, much like the time complexity, the space complexity is also incredibly inefficient.

3.2 The Dynamic Approach

3.2.1 Algorithm Explanation

As with the brute-force algorithm, it begins by using a function from `Utilities.py` to initialise the starting variables. It then creates a 2D array, which is of size n (number of events) by m (maximum available budget), where every cell initially contains 0. Then, using a nested loop, it iterates through every cell in the array. Each cell is considered its own subproblem where it calculates the greatest enjoyment possible given the maximum budget(m) and the first (n) events in the event list. The key part of this optimization is using the values from previous cells to calculate its own value to reduce redundant calculation. It does this by filling the cell with the enjoyment value from the previous cell or the n th event's enjoyment plus the enjoyment value from the previous row for the remaining budget, selecting whichever is greater. For example, if a cell in the array is located at (3, 100), then the value calculated inside that cell represents the maximum enjoyment value that can be obtained with the first 3 available events in the list and 100 maximum budget. This part of the algorithm is the program's use of memoization, as it uses previously calculated values to make further calculations, removing the need to repeat computations that it has already completed, massively improving the approach's efficiency.

Finally, once the entire 2D array has been calculated fully, it selects the cell with the current constraints of the problem, which will be at the bottom corner of the grid. Here, the algorithm begins another loop to deduce the list of events responsible for the maximal solution. The current cell, i , is compared to the cell above it. The cell i shows the enjoyment value considering the first i events while the cell $i-1$ shows the enjoyment value considering the first $i-1$ events. If they have the same value, it means that they have the same set of events so, since event i couldn't have been included in cell $i-1$ to begin with, then event i can't be in cell i either. It then repeats, reducing the number of considered events from the first i events to the first $i-1$ events, selecting a new cell in the grid. This new cell will have a new max budget equaling the old max budget minus the cost of the removed event **only if** the event was a part of the solution. The loop fully concludes once it runs out of events of the max budget reaching 0 suggesting that the solution list uses up all of the budget. At that point, the list is returned, except if no event could fit within the constraint, in which case, the placeholder value of event 0 (signifying no event) is used. The solution is returned as a formatted string to the command line, in the same style as the brute-force algorithm's result.

3.2.2 Pseudocode

Below is a screenshot of the pseudocode for the dynamic-approach algorithm, as to provide an alternative representation of how the algorithm works:

```
# Event Planner Dynamic Pseudocode

function Event_Planner_Dynamic:
    data_field_2d = [[0 for _ in range(max_budget + 1)] for _ in range(number of events + 1)]

    for i in range(n+1)

        for m in range(max_budget+1)

            if i is 0 or m is 0

                set data_field_2d[i][m] to 0
            END if
            else if event_list[i-1][1] is less than or equal to m:

                data_field_2d[i][m] = max of:
                    data_field_2d[i-1][m]
                    and
                    event_list[i-1][2] + data_field_2d[i-1][m-event_list[i-1][1]]

            END else if
            else:

                set data_field_2d[i][m] to data_field_2d[i-1][m]

            END else

        END for

    END for

    i is number_of_events
    j is max_budget

    while x and y is greater than 0

        if data_field_2d[x][y] is not data_field_2d[x-1][y]

            solution is solution + [event_list[x-1]]
            j is j - event_list[x-1][1]

        END if

        decrement x by 1

    END while

    return best_combo, details

END function Event_Planner_Dynamic
```

3.2.3 Time/Space Complexity

The time complexity of the dynamic approach is $O(nm)$, where n is the number of events and m is the max budget; it has a linear time complexity. This specific big O notation is obtained by identifying the size of the 2D array used, which is essentially an n by m grid. Generally, this is sufficient in terms of efficiency, proving particularly so for large values of n .

The space complexity of the dynamic approach is also $O(nm)$. This is the case because the greatest factor impacting the memory used by this program is the initial 2D list of all possible enjoyment values, which is of dimensions n by m . Similar to its time complexity, the dynamic approach has a linear space complexity, meaning that it is considerably more space efficient than the brute-force approach.

4. Testing and Results

4.1 Test Description

For our testing, we used the `input_small.txt`, `input_medium.txt` and `input_large.txt` files, which represents 5, 12 and 25 events respectively. Running these files through both the brute-force and dynamic algorithms allowed us to get a good idea of how the program performed and how efficiently it scaled with larger input sizes.

Furthermore, for the dynamic algorithm, we also tested it with much larger input files, with 100, 200, 500 and even 1000 possible events to choose from. This is because the dynamically programmed algorithm was efficient enough to compute these event lists in a reasonable amount of time, with its polynomial time complexity. This is unlike the brute-force algorithm which wouldn't even be close to handling as many as 100 events (let alone 1000) due to its exponential time complexity. This testing with larger input files permitted us to obtain even more useful information regarding the dynamic algorithms' performance and ability to deal with scaling input size.

4.2 Test Results

4.2.1 Brute-force Algorithm Results

Firstly, we tested the brute-force algorithm with `input_small.txt`, where it found the solution in 0.001 seconds, as shown below:

```

=====Brute Force=====
Input: input_small.txt

-----Solution-----

- Campus-Tour
- Game-Night
- Pizza-Workshop
- Hiking

-----Stats-----

Money used:      190 Pounds
Total enjoyment: 410 Enjoyment
Time to Compute: 0.001402100082486868 Seconds

=====

```

Next, with input_medium.txt, the algorithm calculated the solution in 0.04 seconds, shown below:

```

=====Brute Force=====
Input: input_medium.txt

-----Solution-----

- Welcome-BBQ
- Karaoke-Night
- Film-Screening
- Sports-Tournament
- Pub-Quiz
- Cooking-Class
- Open-Mic

-----Stats-----

Money used:      300 Pounds
Total enjoyment: 565 Enjoyment
Time to Compute: 0.044459199998527765 Seconds

=====

```

Finally, with input_large.txt, it executed in 270.29 (2.d.p) seconds (roughly 4 and a half minutes). This huge spike in execution time is clearly explained by the algorithms' exponential time complexity – this approach scales very poorly as input size begins to significantly increase. The test can be seen below:


```
=====Brute Force=====
Input: input_large.txt

-----Solution-----

- Orientation-Walk
- Ice-Breaker-Games
- Movie-Marathon
- Charity-Run
- Trivia-Night
- Campus-Scavenger-Hunt
- Photography-Walk
- Poetry-Slam
- Dance-Workshop
- Baking-Competition
- Board-Game-Cafe
- Volunteering-Event
- Park-Picnic
- Stargazing-Trip
- Crafts-Fair

-----Stats-----

Money used:      500 Pounds
Total enjoyment: 1135 Enjoyment
Time to Compute: 429.7821565999184 Seconds

=====
```

4.2.2 Dynamic Algorithm Results

As with the brute-force algorithm, we began by testing the dynamic algorithm with input_small.txt. This executed in 0.001 seconds, as shown below:

```

=====Dynamic=====
Input: input_small.txt

-----Solution-----

- No Event
- Campus-Tour
- Game-Night
- Pizza-Workshop
- Hiking

-----Stats-----

Money used:      190 Pounds
Total enjoyment: 410 Enjoyment
Time to Compute: 0.0010349000804126263 Seconds

=====

```

For input_medium.txt, the dynamic algorithm found the solution in 0.002 seconds, shown below:

```

=====Dynamic=====
Input: input_medium.txt

-----Solution-----

- No Event
- Welcome-BBQ
- Karaoke-Night
- Film-Screening
- Sports-Tournament
- Pub-Quiz
- Cooking-Class
- Open-Mic

-----Stats-----

Money used:      300 Pounds
Total enjoyment: 565 Enjoyment
Time to Compute: 0.0024188999086618423 Seconds

=====

```

For the final main test, with input_large.txt, the algorithm executed in 0.007 seconds, as shown below. Already, the performance between the two algorithms is drastically improved:

```

=====Dynamic=====
Input: input_large.txt

-----Solution-----

- No Event
- Orientation-Walk
- Ice-Breaker-Games
- Movie-Marathon
- Charity-Run
- Trivia-Night
- Campus-Scavenger-Hunt
- Photography-Walk
- Poetry-Slam
- Dance-Workshop
- Baking-Competition
- Board-Game-Cafe
- Volunteering-Event
- Park-Picnic
- Stargazing-Trip
- Crafts-Fair

-----Stats-----

Money used:      500 Pounds
Total enjoyment: 1135 Enjoyment
Time to Compute: 0.007018700009211898 Seconds

=====

```

As I mentioned previously, once we realised that the performance of the dynamic algorithm was scaling very well with increased input size, we carried out further tests on 100, 200, 500 and even 1000 available events. Please note that the following screenshots are only smaller as the solution event list is too long to reasonably fit in the image.

For 100 events, it executed in 0.045 seconds:

```

- Business-Debate-81
- Activity-87
- Culture-Panel-89
- Activity-99

-----Stats-----

Money used:      914 Pounds
Total enjoyment: 2671 Enjoyment
Time to Compute: 0.04485049983486533 Seconds

=====

```

For 200 events, it executed in 0.20 seconds:

```

- Charity-Social-188
- Activity-189
- Charity-Game-190
- Photography-Dance-192

-----Stats-----

Money used:      2265 Pounds
Total enjoyment: 5660 Enjoyment
Time to Compute: 0.19827569997869432 Seconds

=====

```

For 500 events, it executed in 0.73 seconds:

```

- Activity-488
- Film-Workshop-496
- Adventure-Tour-497
- Leadership-Sport-499

-----Stats-----

Money used:      3231 Pounds
Total enjoyment: 9767 Enjoyment
Time to Compute: 0.7292188999708742 Seconds

=====

```

For 1000 events, it executed in 2.1 seconds:

```

- Fitness-Walk-975
- Activity-983
- Career-Sport-984
- Adventure-Exhibition-999

-----Stats-----

Money used:      4610 Pounds
Total enjoyment: 15981 Enjoyment
Time to Compute: 2.1199890000279993 Seconds

=====

```

4.3 Performance Comparisons

From the tests in 4.2.1 and 4.2.2, it is clear to see that the dynamic algorithm is not only magnitudes faster than the brute-force algorithm, but it also scales significantly better with increasing input size, which is a vital factor to consider when measuring performance of algorithms. Whilst both algorithms were perfectly capable of computing the input_small.txt and input_medium.txt test files in a perfectly reasonable amount of time (sub 1/20 of a second), the brute-force algorithm saw an unreasonably significant increase

in execution time with the input_large.txt file (taking over 4 minutes to compute) whilst the dynamic algorithm completes this same test in under 1/100 of a second. In fact, the difference in scalability between the two algorithms is so great that not only is the brute-force algorithm effectively incapable of running the 200+ event tests in even a remotely reasonable time, but the dynamic algorithm can also compute 1000 events orders of magnitude faster than the brute-force algorithm can calculate even 25.

This difference in scalability is simply explained by reviewing the time complexities of the two approaches. The linear time complexity of the dynamic algorithm means for magnitudes better scalability (in terms of execution time) when compared to the exponential time complexity of the brute-force algorithm.

In conclusion, the tests distinctly indicate that the dynamic approach to solving this problem is significantly more efficient than the brute-force approach, highlighted clearly by the difference in scalability with an increasing number of available events.

5. Optional Extensions

5.1 Extension 1: Multiple Constraints

5.1.1 Overview & Explanations

We decided to take on the challenge of creating altered versions of both our brute-force and dynamic algorithms which would be capable of simultaneously considering both restraints, those being time and money, when calculating the optimal set of events to maximise the enjoyment value obtained.

This involved developing unique adaptations to different approaches. For the brute force algorithm, this was very simple, as it only required us to ensure that time was also being considered when determining the valid combinations to add to the list to process. For the dynamic algorithm, we found a slightly more complicated but somewhat elegant solution. By changing the 2D array (which stores all possible enjoyment values) into a 3D array, we could associate the additional dimension in terms of time. This meant that instead of any given enjoyment value being indexed by just the maximum number of available events and money available to obtain it, the maximum time available could also be used here as well, thanks to the newly introduced dimension in the array. Since the core of the approach is still the same, we can also continue making use of memoization, in the same way as before, maintaining this algorithms' efficiency. Pseudocode explaining how this dual-restraint dynamic approach works can be seen below:

```

1  # Event Planner Dynamic Pseudocode
2
3  function Event_Planner_Dynamic:
4      data_field_3d = [[[0 for _ in range(max_budget + 1)] for _ in range(max_time + 1)] for _ in range(number of events + 1)]
5
6      for i in range(n+1)
7
8          for t in range(max_time+1)
9
10             for m in range(max_budget+1)
11
12                 if i is 0 or t is 0 or m is 0
13
14                     set data_field_3d[i][t][m] to 0
15                 END if
16                 else if event_list[i-1][1] is less than or equal to t and event_list[i-1][2] is less than or equal to m:
17
18                     data_field_3d[i][t][m] = max of:
19                         data_field_3d[i-1][t][m]
20                     and
21                         event_list[i-1][3] + data_field_3d[i-1][t-event_list[i-1][1]][m-event_list[i-1][2]]
22
23                 END else if
24                 else:
25
26                     set data_field_3d[i][t][m] to data_field_3d[i-1][t][m]
27
28                 END else
29
30             END for
31
32         END for
33
34     END for
35
36     while x and y and z is greater than 0
37
38         if data_field_3d[x][y][z] is not data_field_3d[x-1][y][z]
39
40             solution is solution + [event_list[x-1]]
41             j is j - event_list[x-1][1]
42             k is k - event_list[x-1][2]
43
44         END if
45
46         decrement x by 1
47
48     END while
49
50     return best_combo, details
51
52 END function Event_Planner_Dynamic
53

```

To measure the efficiency of these algorithms, now that they could consider both restraints rather than just money, we carried out the same testing process, which can be seen in 5.1.2 and 5.1.3 below.

5.1.2 Dual-Restraint Brute-force Approach Tests

Testing with the input_small.txt file, the optimal solution was calculated by the dual-restraint brute-force algorithm in 0.001 seconds, as shown in the image below:

```

=====Brute Force=====
Input: input_small.txt

-----Solution-----

- Game-Night
- Pizza-Workshop
- Hiking

-----Stats-----

Time used:      10 Hours
Money used:     170 Pounds
Total enjoyment: 360 Enjoyment
Time to Compute: 0.0011037001386284828 Seconds

=====

```

Testing with the input_medium.txt file, the optimal solution was calculated in 0.045 seconds, as shown below:

```

=====Brute Force=====
Input: input_medium.txt

-----Solution-----

- Film-Screening
- Sports-Tournament
- Art-Workshop
- Pub-Quiz
- Laser-Tag
- Open-Mic

-----Stats-----

Time used:      15 Hours
Money used:     295 Pounds
Total enjoyment: 535 Enjoyment
Time to Compute: 0.045140800066292286 Seconds

=====

```

Finally, testing with the input_large.txt file, the optimal solution took massively longer to compute in comparison to the other two test files, printing the results to the command line after roughly 387 seconds. This is a clear indication that the performance of the brute-force algorithm scales incredibly poorly with increasing input size, not just whilst considering money, but whilst considering time at the same time too.

```

=====Brute Force=====

Input: input_large.txt

-----Solution-----

- Orientation-Walk
- Wine-Tasting
- Rock-Climbing
- Pottery-Class
- Campus-Scavenger-Hunt
- Dance-Workshop
- Comedy-Show
- Crafts-Fair

-----Stats-----

Time used:      20 Hours
Money used:     500 Pounds
Total enjoyment: 840 Enjoyment
Time to Compute: 387.0880841999315 Seconds

=====

```

5.1.3 Dual-Restraint Dynamic Approach Tests

Firstly, testing the dual-restraint dynamic algorithm with the input_small.txt file computed the optimal solution in 0.005 seconds, as shown by the image below:

```

=====Dynamic=====

Input: input_small.txt

-----Solution-----

- No Event
- Game-Night
- Pizza-Workshop
- Hiking

-----Stats-----

Time used:      10 Hours
Money used:     170 Pounds
Total enjoyment: 360 Enjoyment
Time to Compute: 0.005089500220492482 Seconds

=====

```

Next, testing with the input_medium.txt file, the algorithm computed the optimal solution in 0.025 seconds, shown below:


```

=====Dynamic=====
Input: input_medium.txt

-----Solution-----

- No Event
- Film-Screening
- Sports-Tournament
- Art-Workshop
- Pub-Quiz
- Laser-Tag
- Open-Mic

-----Stats-----

Time used:      15 Hours
Money used:     295 Pounds
Total enjoyment: 535 Enjoyment
Time to Compute: 0.025850199861451983 Seconds

=====

```

Testing with the input_large.txt file, the algorithm computed the optimal solution in 0.12 seconds. Even with the largest main test file, this dynamic approach solves the problem in under 1/5 of a second, despite additionally now considering both money and time simultaneously:

```

=====Dynamic=====
Input: input_large.txt

-----Solution-----

- No Event
- Orientation-Walk
- Wine-Tasting
- Rock-Climbing
- Pottery-Class
- Campus-Scavenger-Hunt
- Dance-Workshop
- Comedy-Show
- Crafts-Fair

-----Stats-----

Time used:      20 Hours
Money used:     500 Pounds
Total enjoyment: 840 Enjoyment
Time to Compute: 0.12487470009364188 Seconds

=====

```

Now we move onto the larger tests, as before. For 100 events, it took 2.5 seconds:

```
- Design-Competition-70
- Business-Debate-81
- Activity-87
- Activity-99

-----Stats-----

Time used:      50 Hours
Money used:     908 Pounds
Total enjoyment: 2212 Enjoyment
Time to Compute: 2.5006652001757175 Seconds

=====
```

For 200 events, it took 16.86 seconds:

```
- Culture-Social-177
- Literature-Debate-181
- Community-Game-186
- Photography-Dance-192

-----Stats-----

Time used:      67 Hours
Money used:     2264 Pounds
Total enjoyment: 4083 Enjoyment
Time to Compute: 16.858194099972025 Seconds

=====
```

For 500 events, it took 79.87 seconds:

```
- Science-Seminar-494
- Career-Meeting-495
- Film-Workshop-496
- Charity-Competition-498

-----Stats-----

Time used:      81 Hours
Money used:     3230 Pounds
Total enjoyment: 5974 Enjoyment
Time to Compute: 79.87068920000456 Seconds

=====
```

Finally, for 1000 events, it took 405 seconds. Considering the 1000 available events for this test, computing the optimal solution at this speed is very impressive and clearly demonstrates, alongside the other tests, that the performance of the dynamic algorithm scales incredibly well with increasing input size.

```
- Activity-946
- Charity-Festival-949
- Fitness-Walk-975
- Language-Debate-977

-----Stats-----

Time used:      138 Hours
Money used:     4610 Pounds
Total enjoyment: 9190 Enjoyment
Time to Compute: 405.2863077998627 Seconds

=====
```

5.1.4 Results and Conclusions

As expected, these dual-restraint algorithms calculate solutions, overall, slower than the single-restraint algorithms, as there are generally fewer operations to execute. Despite this, however, many tests indicate that our modified algorithms are generally successful, particularly the dynamic algorithm, which we effectively adapted to handle multiple restraints at the same time. Even if it may struggle with processing 500+ events, it is perfectly capable of finding the optimal solution for scenarios with 100 events or less in a reasonable timeframe.

To confirm that these new algorithms indeed consider both restraints, we can check the difference between the solution found by each algorithm's money-only version and both-restraints version.

Here, comparing the brute force algorithms for both shows that the difference in the limits changes what the maximal solution is. Although they both use the same amount of money, the list of events that make up the winning solution and the enjoyment values are different, suggesting that the algorithm has taken the additional constraint into account.

```

=====Brute Force=====
Input: input_large.txt

-----Solution-----

- Orientation-Walk
- Ice-Breaker-Games
- Movie-Marathon
- Charity-Run
- Trivia-Night
- Campus-Scavenger-Hunt
- Photography-Walk
- Poetry-Slam
- Dance-Workshop
- Baking-Competition
- Board-Game-Cafe
- Volunteering-Event
- Park-Picnic
- Stargazing-Trip
- Crafts-Fair

-----Stats-----

Money used:      500 Pounds
Total enjoyment: 1135 Enjoyment
Time to Compute: 429.7821565999184 Seconds

=====

```

```

=====Brute Force=====
Input: input_large.txt

-----Solution-----

- Orientation-Walk
- Wine-Tasting
- Rock-Climbing
- Pottery-Class
- Campus-Scavenger-Hunt
- Dance-Workshop
- Comedy-Show
- Crafts-Fair

-----Stats-----

Time used:      20 Hours
Money used:      500 Pounds
Total enjoyment: 840 Enjoyment
Time to Compute: 387.0880841999315 Seconds

=====

```

The difference in solutions is also clearly observed in the two dynamic approaches. Using input_medium.txt, the single-restraint version uses £300 for its optimal solution, whilst the dual restraint version uses only £295. This can be seen below as before:

```
=====Dynamic=====
Input: input_medium.txt

-----Solution-----

- No Event
- Welcome-BBQ
- Karaoke-Night
- Film-Screening
- Sports-Tournament
- Pub-Quiz
- Cooking-Class
- Open-Mic

-----Stats-----

Money used:      300 Pounds
Total enjoyment: 565 Enjoyment
Time to Compute: 0.0024188999086618423 Seconds

=====
```

```
=====Dynamic=====
Input: input_medium.txt

-----Solution-----

- No Event
- Film-Screening
- Sports-Tournament
- Art-Workshop
- Pub-Quiz
- Laser-Tag
- Open-Mic

-----Stats-----

Time used:      15 Hours
Money used:      295 Pounds
Total enjoyment: 535 Enjoyment
Time to Compute: 0.025850199861451983 Seconds

=====
```

The performance of these modified, dual-restraint algorithms can be effectively measured by identifying and analysing how their respective results have changed from their original single-restraint versions. As neither of the algorithms had to be drastically changed, at least not in terms of the core of their approach, the brute-force approach remained exponential with its time and space complexity (since every possible combination is still being calculated) and the dynamic approach remained linear with its time and space complexity as well. However, it is crucial to note that, despite the general complexity of the dynamic approach staying the same, it does actually change from $O(nm)$ to $O(nmt)$ as time is now included as another dimension of the grid, hence impacting the number of operations to perform and memory required. This is why the algorithm still scales similarly but overall executes slower. It should also be stated that, once again, the dynamic approach scales much better than the brute-force approach as input size begins to significantly increase.

6. References

- The only notable source was freeCodeCamp.org, which is an educational coding platform/community which aided with the research and development of the dynamic approach.